COM3505 IoT System Technical Report
ESP32 Web Dashboard with LEDs

Jakub Bala, Daniel Voice [Team Microslop]

1. System Overview

The goal of this IoT project is to create an integrated system that uses the ESP32-S3 microcontroller and a web server to allow a user to access sensor readings and control LED outputs over a local WiFi network. Alongside our own computers and WiFi network, we used the following electronics hardware:

- 1x Half Breadboard with jumper cables
- 1x Adafruit ESP32-S3 Feather microcontroller
- 1x TMP36 temperature sensor
- 6x LEDs (2x red, 2x yellow, 2x green)
- 6x 120Ω resistors

In order to establish communication with the ESP32, we used a Flask web server with a /sensor endpoint. The ESP32 performs periodic telemetry of ambient temperature, pushing data to the server via HTTP POST requests every two seconds. The server 'piggybacks' off this request, sending any pending command to change the LED pattern in the response. The user interacts with the system via a typical frontend webpage (HTML, CSS, JS). The webpage uses AJAX to communicate with multiple server endpoints. AJAX makes requests in the background, enabling live updates of interactive features:

- Live temperature sensor updates
- Interactive graph with temperature reading history (Last hour, Last 24 hours, Last 7 days)
- Selection from multiple light pattern options
- Web server and ESP connection status indicators

This architecture enables a data flow from ***sensor → ESP32 → Flask server → browser*** and an opposite control flow from ***browser → Flask → ESP32 → LEDs.***

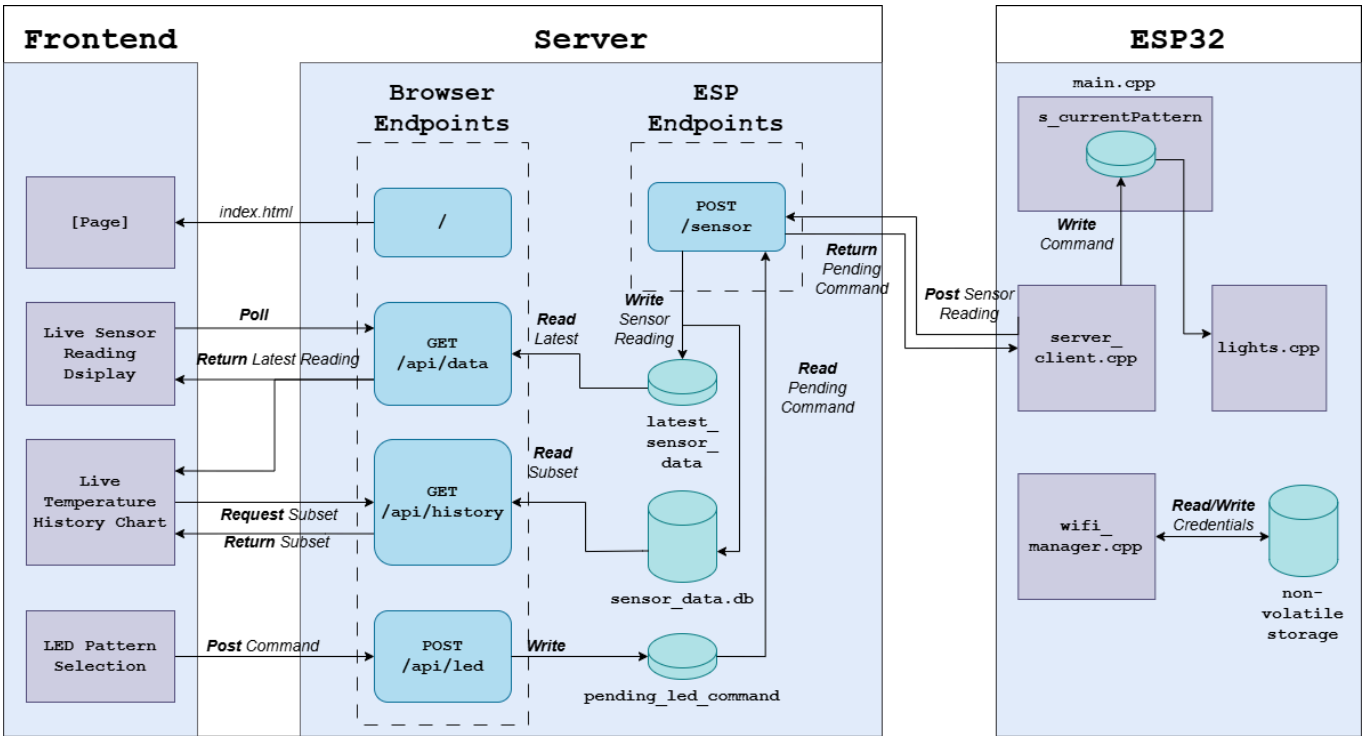

Figure 1: System architecture showing data and control flow between ESP32, Flask server, and web browser.

2. Hardware Setup

As shown in figure 2, we made use of the breadboard's capabilities and carefully considered our circuit design, aiming for compactness, modularity and effective use of the ESP's GPIO pins. GPIO13 is used for temperature readings from the temperature sensor's **Vout** pin (yellow wire). GPIO 12,11,10,9,6 and 5 are used for LED outputs (green wire). These GPIO pins are all located contiguously on the side of the microcontroller and offer some advantages:

- Full digitalWrite/digitalRead support (can go HIGH and LOW for LED control)
- No strapping function (won't interfere with boot)

We connected the microcontroller's **GND** pin to the **GND(-)** rail on the breadboard and terminated the LED and sensor circuits there. The **3.3V** pin on the ESP was used to power the temperature sensor's **+Vs** pin as this requires a consistent, reliable supply of power. Red wires were used for power, black for GND and yellow for sensor output as this is standard convention in electronics, and improves circuit understandability.

Using 120Ω resistors means the amperage is kept within the LEDs rated range ((3.3V − ~2.0Vf) / 120Ω ≈ 10.8mA, well within typical 20mA range) making the circuit capable of long periods of use.

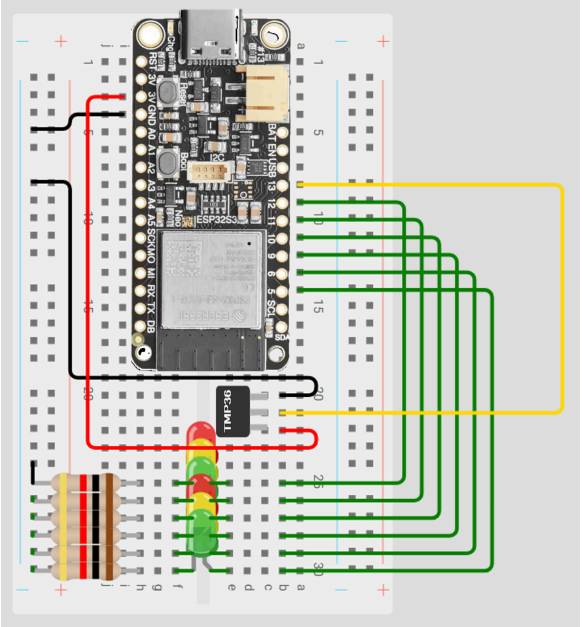

Figure 2: Hardware wiring diagram showing LED and sensor connections to the ESP32.

3. ESP32 Firmware

3.1 Wi-Fi Connectivity & Provisioning

Our network provisioning system avoids impractical hardcoded credentials by storing network credentials in Non Volatile Storage (NVS). On boot, the ESP checks NVS. If empty, it enters **PROVISIONING** mode and starts a Wireless Access Point (WAP). If credentials exist, the ESP attempts to connect. Success triggers **NORMAL** mode, enabling sensor data POSTing and listening for LED commands. If the connection attempt times out, the ESP wipes the NVS and reverts to **PROVISIONING** mode, starting a WAP.

When provisioning, the ESP serves a network called 'ESP32-Setup'. Mobile users connecting to this network will be automatically redirected to a simple captive portal served on `192.168.4.1` showing available networks. The user can select a network and enter credentials, which will start a wifi.Begin() attempt. If successful, the credentials are saved to NVS and the ESP restarts.

3.2 LED Pattern Algorithms

We implemented five different LED patterns:

- **Chase** - Sequential LED sweep, bounces back and forth rather than wrapping.
- **Blink** - Synchronous on/off, all LEDS.
- **Alternating(Rainbow)** - Colour-grouped cycling (red -> green -> yellow)
- **Flicker** - Probabilistic flame simulation with realistic behaviour
- **Twinkle** - Random on/off flickering for each LED

To improve modularity and readability, we moved the light pattern logic into **lights.cpp**. We replaced hard-coded pin values by defining LED arrays in **lights.h** and making use of helper functions **allOn()**, **allOff()**, **arrayOn()** and **arrayOff()**. This abstraction ensures that hardware changes, such as moving or adding LEDs, require minimal code updates.

The most noteworthy pattern is **flicker**. This pattern simulates the real behaviour of a candle flame as it is 'lit' from the bottom of the LEDs array (the start). Each LED cannot light unless the one before it is lit and a random value exceeds a certain threshold. The threshold is higher for LEDs further in the array, meaning the flame only occasionally flickers to the 'top'.

3.3 Sensor Integration

The temperature sensor is a TMP36 sensor, which outputs an analog signal via the Vout pin. This voltage signal scales linearly with temperature, according to the following equation:

`Vout = 500mV + (10mV × temp_in_C)`

In order to transform the analog signal at the GPIO pin to a float value for temperature, the ESP firmware performs the following three steps, found at `server_client.cpp:line22`:

1. analogRead on temperature pin, obtains value from 0-4095:
   `int raw = analogRead(TEMP_PIN);`
2. Divide by 4095 to get a 0-1 fraction, then multiply by 3.3 to get `Vout`:
   `float voltage = (raw / 4095.0f) * 3.3f;`
3. Apply the signal equation, solving for `temp_in_C`:
   `float temperature = (voltage - 0.5f) / 0.01f;`

This float value is passed to the Flask server in the next POST request (every 2000ms).

4. Web Server & Dashboard

4.1 Flask Server

The server utilizes simple in-memory variables (*latest_sensor_data* and *pending_led_command*). This in-memory state is sufficient for a single-device IoT system, as there is no need for complex ID routing or heavy database queries.

Each incoming reading from the ESP is stored in *latest_sensor_data*, and written to the *sensor_data.db* persistent SQLite database. When a user selects a new LED pattern in the dashboard, it is stored in the *pending_led_pattern* variable. To minimize HTTP requests from the microcontroller, the server utilizes a "piggyback" delivery method: When the ESP POSTs to /sensor with its temperature data, the server checks if there is a pending LED command, and if there is, it is attached to the HTTP 200 OK response. The server then clears the pending command to prevent the ESP continuously restarting the pattern. The piggybacking pattern means that the ESP only ever makes outbound requests, so it does not need to host its own endpoint for receiving commands.

The server exposes the /api/history endpoint, which the dashboard can request in order to chart the temperature readings over a period of time. Bucketing logic is used in order to avoid sending thousands of raw data points. Depending on the requested time frame (1 hour, 24 hours, 7 days), the server groups the timestamps into specific intervals (1 minute, 15 minute, 6 hour). This ensures that the dashboard receives a clean and evenly spaced dataset, as well as preventing chart rendering lag.

To prevent the SQLite database growing indefinitely, a BackgroundScheduler from the ‘apscheduler’ package is used to schedule the database for cleaning every hour. When this happens, any data older than 7 days is erased.

4.2 Web Dashboard

The dashboard consists of a live temperature reading panel, LED control panel, and a live-updating historical temperature chart, arranged in a card-based grid. A responsive breakpoint at 1000px allows the dashboard to be accessed on a mobile device, where the cards become arranged as a vertical stack.

The dashboard polls the /api/data server endpoint every second for the most recent temperature reading and updates the display without requiring a refresh of the page.

Connection badges are visible in the top right corner of the dashboard which convey whether the server and ESP are accessible. If the ESP poll returns as *null*, the ESP is assumed to be inaccessible. If the server /api/data endpoint is unable to be reached, the server is assumed to be inaccessible (implicitly meaning the ESP is also inaccessible). If either the server or ESP is inaccessible, the badge goes grey and says “[Server / ESP] offline”

Chart.js is used to display the historical temperature readings chart. When the dashboard is first accessed in a session, the chart will display the readings from the last hour. There is a dropdown selection to change the chart between showing the last hour, last 24 hours, or last 7 days of temperature readings. When a sensor reading is received, the reading is appended to the chart instead of sending constant heavy history requests while keeping the chart live-updating.

The LED control panel contains a button for each of the 5 LED modes. When an LED pattern is selected, a HTML attribute corresponding to the chosen pattern is POSTed to the server’s /api/led endpoint. The selected button changes its style to provide visual feedback as to which pattern is selected while the server passes the command on to the ESP.

5. Architecture Choices

The ESP32 sends a JSON POST every 2 seconds over HTTP to the /sensor server endpoint. 2 second intervals were chosen to ensure bandwidth isn’t being wasted while readings are still frequent. The Flask server stores the reading in the *sensor_data.db* persistent SQLite database, as well as the *latest_sensor_reading* in-memory variable. By using a persistent database the dashboard is able to display historical temperature data even if the server restarts. Using an in-memory variable for the latest sensor reading removes the need for the server to query the database each time the dashboard polls sensor data.

The dashboard uses HTTP AJAX to poll the /api/data server endpoint every second (1000ms). The dashboard polling the server at double the rate that the ESP sends data to it means that due to Niquist’s theorem, the data that the dashboard displays will always be the most recent reading. HTTP polling was used instead of WebSockets as implementation is simpler, and the project does not require extremely low latency between elements in order to remain effective.

Figure 3 - Status Indicators on browser

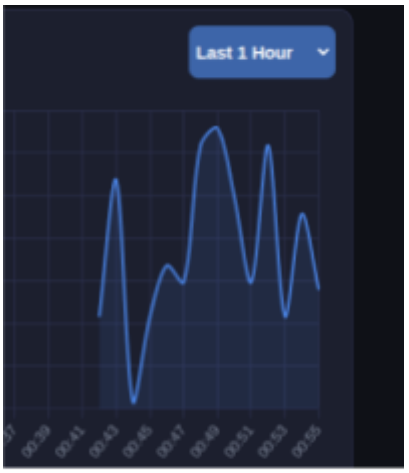

Figure 4 - Temperature readings chart

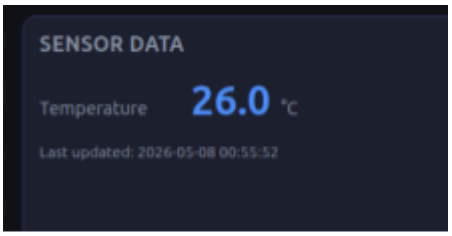

Figure 5 - Most recent temperature reading